33

RAG

Retrieval-Augmented Generation

Give an LLM your documents so it answers from facts

AI/ML Engineer Learning Series · Deep Dive Edition

What it is	Letting an LLM answer using YOUR documents
Full name	Retrieval-Augmented Generation
The trick	Find relevant text, then let the LLM use it
Fixes	Hallucination, private data, fresh info
Your project	Exactly what your PDF RAG Chatbot does

What's Inside

Here is everything covered in this guide, in order:

#	Section	What you'll learn
1	**The Problem RAG Solves**	Why a plain LLM isn't enough
2	**What is RAG?**	Retrieve, then generate
3	**The Full Pipeline**	Every step, start to finish
4	**Chunking**	Splitting documents wisely
5	**Retrieval**	Finding the right pieces
6	**Augment and Generate**	The LLM writes the answer
7	**Hybrid Retrieval**	Meaning + keywords (your project)
8	**RAG in Code**	The whole thing, simplified
9	**Making RAG Better**	Practical improvements
10	**Common Mistakes**	What to avoid
★	**Cheatsheet**	Quick reference

1. The Problem RAG Solves

LLMs are brilliant, but they have three big limits you learned earlier: they hallucinate (make things up), they don't know your private documents, and their knowledge has a cutoff date. Ask a plain LLM 'what does MY company handbook say about leave?' and it simply can't know — it was never trained on your handbook.

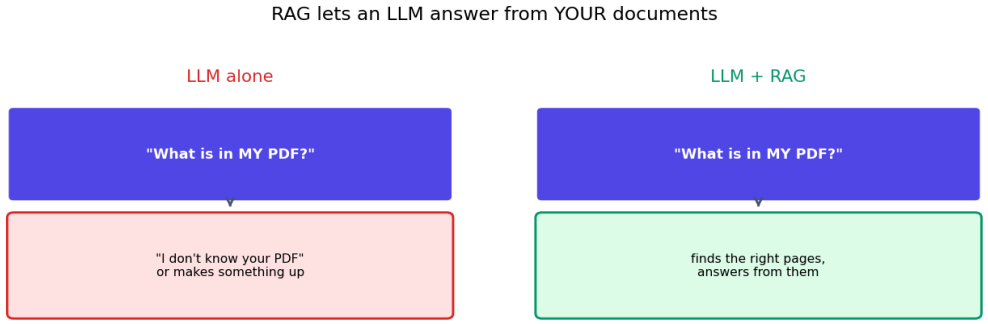

Fig 1 — A plain LLM can't answer about your documents. RAG finds the right text first, so it can.

RAG fixes all three. The idea is beautifully simple: before the LLM answers, you FIND the relevant information from your own documents and HAND it to the model. Now the LLM answers from real, specific facts instead of guessing. This is the single most important pattern in applied AI today — and exactly what your PDF chatbot does.

2. What is RAG?

RAG stands for Retrieval-Augmented Generation. The name is literally the three steps:

- **Retrieval** — find the most relevant pieces of text from your documents.
- **Augmented** — add those pieces to the LLM's prompt as context.
- **Generation** — the LLM generates an answer using that context.

In plain words: look up the right information, give it to the LLM, let the LLM write the answer. It combines everything you've learned — embeddings, vector search, and LLMs — into one powerful system. The LLM provides the language skill; your documents provide the facts.

> ■ *Think of it like an open-book exam. A plain LLM answers from memory (and may misremember). RAG lets it open the book to the right page first, then answer. Far more accurate and trustworthy.*

3. The Full Pipeline

Here's the complete RAG system, tying together every topic from this stage. It has two phases: a one-time setup, and the per-question answer flow.

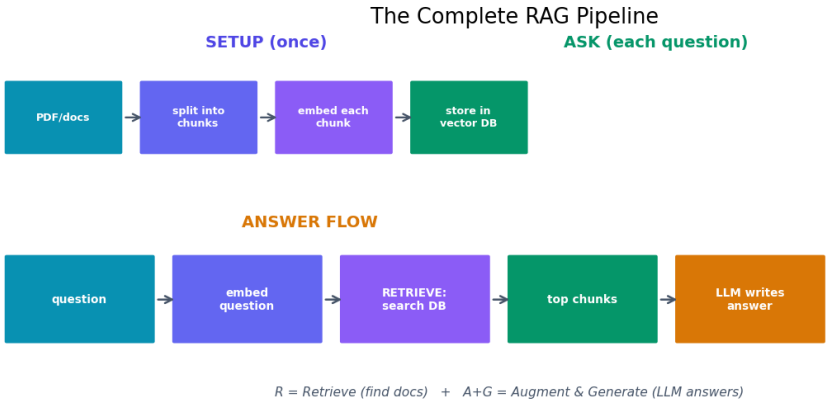

Fig 2 — The full RAG pipeline. Setup happens once; the answer flow runs for every question.

Setup (done once)

- Take your documents (like a PDF).
- Split them into small chunks.
- Embed each chunk into a vector (sentence transformer).
- Store all vectors in FAISS or a vector database.

Answer flow (every question)

- Embed the user's question into a vector.
- Retrieve: search the store for the closest chunks.
- Augment: put those chunks into the LLM's prompt.
- Generate: the LLM writes the answer from them.

4. Chunking

You can't embed a whole 100-page PDF as one vector — it would lose all detail. So first you split documents into small pieces called chunks. Each chunk is a focused bit of text (a paragraph or a few sentences) that can be embedded and searched on its own.

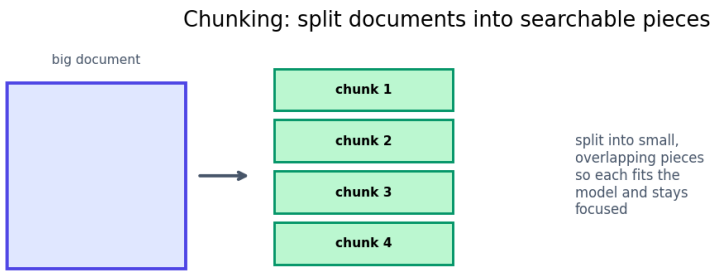

Fig 3 — Chunking: split a big document into small, focused, slightly overlapping pieces.

Good chunking matters a lot. Too big, and a chunk covers many topics, making search fuzzy. Too small, and it loses context. A common choice is a few hundred words per chunk, with a little overlap between chunks so ideas that cross a boundary aren't lost. Getting chunking right is one of the biggest levers for RAG quality.

■ *Overlap is a neat trick: if chunk 1 ends mid-thought and chunk 2 begins there, overlapping them by a sentence or two means the full thought appears in at least one chunk. This avoids losing answers that sit on a boundary.*

5. Retrieval

Retrieval is the 'R' — the heart of RAG. When a question comes in, you embed it and search your stored chunks for the closest matches (using FAISS or a vector database). The top few chunks are the ones most likely to contain the answer.

The quality of retrieval decides everything. If you retrieve the wrong chunks, even the best LLM will give a poor answer — it can only work with what you give it. If you retrieve the right chunks, the answer is usually excellent. This is why so much RAG effort goes into better chunking and retrieval.

Retrieval setting	What it controls
top-k	How many chunks to fetch (e.g. 3-5)
chunk size	How big each searchable piece is
overlap	Shared text between neighbouring chunks
search type	Semantic, keyword, or hybrid

6. Augment and Generate

Once you have the top chunks, you 'augment' the LLM's prompt by inserting them as context, then ask it to answer using ONLY that context. This is where good prompt engineering (Topic 30) matters.

```
prompt = f'''
Answer the question using ONLY the context below.
If the answer is not in the context, say you don't know.

Context:
{retrieved_chunks}

Question: {user_question}
'''

answer = llm(prompt)   # the LLM generates from the context
```

That instruction — 'use only the context, and admit if you don't know' — is the key to trustworthy RAG. It stops the LLM from falling back on its own guesses and keeps answers grounded in your documents. This is the main defence against hallucination.

7. Hybrid Retrieval

Semantic search (by meaning) is powerful but sometimes misses exact terms — like a specific product code or name. Keyword search (like BM25) is great at exact matches but misses meaning. Hybrid retrieval combines both for the best of each.

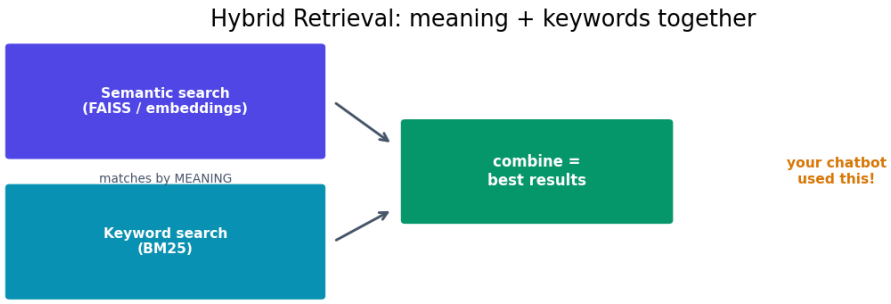

Fig 4 — Hybrid retrieval: semantic search (meaning) plus keyword search (BM25), combined for stronger results.

This is exactly what your PDF RAG Chatbot used: FAISS for semantic matching plus BM25 for keyword matching, combined into one hybrid retriever. Semantic search catches questions phrased differently from the document; keyword search nails exact terms the embeddings might rank too low. Together they retrieve more reliably than either alone — a smart, modern design choice.

> ■ *Hybrid retrieval is a strong portfolio detail. It shows you understand that semantic search isn't perfect and that combining methods gives better results — exactly the kind of practical insight employers look for.*

8. RAG in Code

Here's the whole RAG system in simplified code, pulling together everything you've learned:

```
from sentence_transformers import SentenceTransformer
import faiss, numpy as np

embedder = SentenceTransformer('all-MiniLM-L6-v2')

# --- SETUP (once) ---
chunks = split_into_chunks(my_pdf_text)         # 1. chunk
vectors = embedder.encode(chunks).astype('float32')  # 2. embed
index = faiss.IndexFlatL2(vectors.shape[1])     # 3. store
index.add(vectors)

# --- ANSWER a question ---
def answer(question):
    q_vec = embedder.encode([question]).astype('float32')
    _, ids = index.search(q_vec, k=3)           # 4. retrieve
    context = '\n'.join(chunks[i] for i in ids[0])

    prompt = f'Use only this context:\n{context}\n\n' \
             f'Question: {question}'             # 5. augment
    return llm(prompt)                          # 6. generate
```

9. Making RAG Better

- **Better chunking** — tune chunk size and overlap; split on natural boundaries.
- **Hybrid retrieval** — combine semantic + keyword search (like you did).
- **Retrieve more, then re-rank** — fetch top 20, use a re-ranker to pick the best 3.
- **Good grounding prompt** — 'answer only from context; say if unsure'.
- **Add metadata** — store source, page, date so you can filter and cite.
- **Show sources** — tell the user which chunks the answer came from (builds trust).

■ *Returning the source chunks alongside the answer is a great feature: users can verify where the answer came from, and it makes your chatbot far more trustworthy. A nice addition for your portfolio version.*

10. Common Mistakes

Mistake	Why it's a problem	Fix
Chunks too big or small	Fuzzy or context-less search	Tune size + add overlap
Weak retrieval	LLM gets wrong context	Use hybrid search, re-ranking
No grounding instruction	LLM still hallucinates	'Answer only from context'
Different embed models	Query and docs don't match	Same model for both
Retrieving too few chunks	Misses the answer	Increase top-k a little
Ignoring sources	Users can't verify	Return the source chunks

Quick Reference Cheatsheet

Concept	Meaning
RAG	retrieve docs, then LLM generates from them
Retrieval	find the most relevant chunks
Augment	add chunks to the LLM prompt
Generation	LLM writes the answer from context
Chunking	split docs into small searchable pieces
Overlap	shared text between chunks
top-k	how many chunks to retrieve
Grounding prompt	'answer only from the context'
Hybrid retrieval	semantic + keyword (BM25) search
Fixes	hallucination, private data, stale knowledge

Setup once	chunk -> embed -> store
Per question	embed -> retrieve -> augment -> generate

The one idea to remember: RAG gives an LLM an open book. It retrieves the most relevant chunks from your documents, adds them to the prompt, and the LLM generates an answer grounded in real facts — fixing hallucination and letting it answer about YOUR data.

Next Topic: LlamaIndex — a framework that makes building RAG systems much easier, handling chunking, embedding, storage, and retrieval for you in just a few lines.